



Universidad Experimental Nacional de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

P.F.G: Ingeniería en Informática

Unidad Curricular: Lenguaje y Compiladores

Tema 2 (Los lenguajes de programación)

Profesor:

Marquez Felix

Estudiantes:

Cortez Samuel V- 27.158.842

Duarte Alejandro V-28.530.657

Moreno Alondra V-30.331.870

Perdomo Arnaldo V-30.791.551

Ciudad Guayana, junio 2026

Índice

Resumen Académico.....	3
Introducción.....	4
ACTIVIDAD I: Matriz Descriptiva y Análisis de Paradigmática.....	5
1 Análisis Detallado de Paradigmas Fundamentales y Emergentes.....	5
A. Paradigma Imperativo / Estructural.....	5
B. Paradigma Orientado a Objetos (POO).....	6
C. Paradigma Funcional.....	8
D. Paradigma Lógico / Declarativo.....	9
E. Paradigma Concurrente / Actores (Emergente).....	10
El Fenómeno Contemporáneo de la Convergencia Multiparadigma.....	12
Tabla Comparativa de Integración Multiparadigma.....	12
ACTIVIDAD II: Estudio morfológico, sintáctico y benchmarking en procesamiento intensivo para contrastar las filosofías de diseño de lenguajes con diferentes propósitos y arquitecturas internas.....	14
Análisis de Zig: Morfología, Sintaxis y Benchmarking.....	14
Análisis Morfológico (Léxico) de Zig.....	15
Análisis Sintáctico de Zig.....	16
Análisis e Integración de JavaScript (Node.js).....	18
Análisis Morfológico (Léxico) del Código:.....	20
Análisis Sintáctico y Reglas de Producción Gramatical.....	20
Análisis e Integración de Python.....	23
Análisis morfológico (léxico) del código Python.....	24
Análisis sintáctico del código Python.....	25
Análisis Morfológico de Rust.....	27
Análisis Sintáctico de Rust.....	30
Análisis Sintático del Código Rust.....	32
ACTIVIDAD III: Especificación, Gramática y Operatividad del Lenguaje "L".....	37
Especificación Léxica y Morfológica (Tokens de "L").....	38
Código Fuente de los Programas Operativos.....	47
Evidencia de Ejecución.....	48
Discusión Técnica sobre el Diseño y Seguridad del Lenguaje L.....	49
Conclusión.....	51
Referencias Bibliográficas.....	52

Resumen Académico

El presente informe de investigación, desarrollado en el marco de la asignatura Lenguaje y Compiladores (UNEG, 2026-I), tiene como propósito el análisis comparativo de cuatro lenguajes de programación — Zig, Python, Rust y JavaScript — desde las perspectivas morfológica, sintáctica y de rendimiento computacional. El autor se especializa en el lenguaje Zig, un lenguaje de sistemas de nueva generación diseñado como sucesor directo de C, destacado por su control explícito de memoria, su sistema de metaprogramación en tiempo de compilación (comptime) y la ausencia de comportamiento oculto en tiempo de ejecución.

Como ejercicio de benchmarking, se implementó la demostración de la Conjetura de Collatz para todos los números naturales menores que 1,000,000. Esta conjetura establece que, aplicando iterativamente la secuencia $n/2$ (si n es par) o $3n+1$ (si n es impar), cualquier número entero positivo eventualmente alcanza el valor 1. Los resultados comparativos evidencian la supremacía de los lenguajes compilados nativamente (Zig, Rust) frente a los interpretados o JIT (Python, JavaScript), con Zig completando el cómputo en aproximadamente 411 ms frente a los 12,230 ms de Python.

Como culminación del estudio, el equipo diseñó el Lenguaje L, un Lenguaje de Dominio Específico (DSL) orientado al control y monitoreo del sistema ECO-GRID, una microred eléctrica inteligente con paneles solares, baterías de litio y sensores térmicos. Este ejercicio de diseño lingüístico permitió interiorizar cómo las decisiones léxicas y sintácticas de un lenguaje impactan directamente en la expresividad, seguridad y mantenibilidad del software desarrollado sobre él.

Introducción

En los estadios formativos de la ingeniería en informática, la interacción del estudiante con los lenguajes de programación se circunscribe casi exclusivamente al rol de usuario aplicativo: se parte de una lógica algorítmica preestablecida y se traduce mecánicamente a una sintaxis específica para resolver problemas de complejidad moderada. Sin embargo, al alcanzar el nivel curricular correspondiente al diseño de compiladores e intérpretes, emerge la necesidad epistemológica de trascender esta visión instrumental.

Un lenguaje de programación no es una herramienta estática; es un producto de ingeniería de software rigurosamente diseñado, dotado de una arquitectura formal que abarca tres niveles de análisis interrelacionados: el nivel morfológico (léxico), que define los átomos válidos del lenguaje —los tokens—; el nivel sintáctico, que rige las reglas de combinación de dichos tokens para formar sentencias válidas; y el nivel semántico, que establece el significado de esas construcciones.

El presente informe aborda los paradigmas fundamentales de programación, realiza un análisis morfológico y sintáctico del lenguaje Zig, ejecuta un benchmark de rendimiento computacional mediante la Conjetura de Collatz, y culmina con el diseño de un DSL para sistemas de control de microrredes eléctricas. La relevancia de este estudio radica en que las decisiones de diseño léxico y sintáctico de un lenguaje tienen consecuencias directas y mensurables en la eficiencia del compilador, la expresividad del código resultante y la mantenibilidad del software a largo plazo.

ACTIVIDAD I: Matriz Descriptiva y Análisis de Paradigmática

1 Análisis Detallado de Paradigmas Fundamentales y Emergentes

A. Paradigma Imperativo / Estructural

- **Principios Fundamentales:**

- El programa se concibe como una secuencia de instrucciones que modifican el estado de la memoria.
- Control explícito del flujo mediante estructuras de secuencia, selección (if, switch) e iteración (bucles).
- Variables mutables y la asignación como operación básica del sistema.
- Uso de subprogramas (procedimientos o funciones) para modularizar el código. El enfoque estructural añade la disciplina de bloques y prohíbe el uso indiscriminado de saltos incondicionales (goto).

- **Características Clave:**

- **Estado Mutable:** Los valores de las variables cambian dinámicamente durante la ejecución.
- **Efectos Secundarios:** Las funciones pueden modificar variables globales o parámetros pasados por referencia de forma implícita.
- **Modelo de Ejecución:** Estrecha correspondencia con la arquitectura física de **Von Neumann** (unidad de procesamiento + memoria).

Ventajas	Limitaciones
• Sencillez conceptual y mapeo directo a algoritmos básicos. • Alto rendimiento computacional. • Control fino y directo sobre los recursos de hardware. • Ideal para sistemas operativos y controladores de bajo nivel.	• Alta dificultad para el razonamiento formal debido al estado mutable. • Propenso a errores latentes por efectos secundarios no controlados. • Escalabilidad muy limitada en sistemas concurrentes (condiciones de carrera).

- **Lenguajes Representativos:** C, Pascal, Fortran, Cobol y **Zig** (el cual añade metaprogramación en tiempo de compilación y gestión explícita de memoria sin ocultamientos).

B. Paradigma Orientado a Objetos (POO)

- **Principios Fundamentales:**
 - Organización del software en torno a **objetos**, los cuales encapsulan datos (atributos) y comportamientos (métodos).
 - La comunicación e interacción inter-entidad se produce exclusivamente mediante el **envío de mensajes**.
 - Sustentado firmemente en cuatro pilares: **encapsulamiento, herencia, polimorfismo y abstracción**.

- **Características Clave:**

- **Encapsulamiento:** Ocultación de la información interna y exposición controlada a través de una interfaz pública.
- **Herencia vs. Composición:** Mecanismos para la reutilización de código; la tendencia moderna prioriza la composición para evitar jerarquías de clases rígidas y acopladas.
- **Polimorfismo:** Capacidad de que una misma interfaz posea múltiples implementaciones (sobrecarga, sobrescritura y tipos genéricos).

Ventajas	Limitaciones
• Modelado intuitivo del dominio del problema ("objetos del mundo real"). • Alta mantenibilidad y extensibilidad gracias a la modularidad. • Facilita el trabajo en equipo mediante contratos de interfaz bien definidos.	• Complejidad arquitectónica adicional (diseño de clases y jerarquías). • Riesgo elevado de sobreingeniería en problemas simples. • El estado mutable sigue presente (encapsulado), dificultando la concurrencia nativa.

- **Lenguajes Representativos:** Java, C++, C#, Python, Ruby y Smalltalk.

C. Paradigma Funcional

- **Principios Fundamentales:**
 - La computación se trata como la evaluación de **funciones matemáticas puras**, libres de efectos secundarios.
 - **Inmutabilidad de datos:** Los valores no se modifican *in-place*; cualquier transformación genera un nuevo dato a partir del existente.
 - **Funciones como ciudadanos de primera clase:** Pueden asignarse a variables, pasarse como argumentos y ser devueltas por otras funciones.
 - **Evaluación Perezosa (*Lazy Evaluation*):** Las expresiones retrasan su evaluación hasta que su resultado computacional sea estrictamente necesario.
- **Características Clave:**
 - **Transparencia Referencial:** Una expresión puede ser reemplazada por su valor resultante sin alterar el comportamiento del programa.
 - Uso de la **recursión** como mecanismo primario de iteración.
 - Soporte avanzado de tipos de datos algebraicos y emparejamiento de patrones (*pattern matching*).
 - Aislamiento de efectos colaterales en el mundo exterior mediante abstracciones avanzadas (como las *mónadas* en Haskell).

Ventajas	Limitaciones
• Facilidad para el razonamiento formal y la verificación matemática del código. • Paralelización natural y segura debido a la ausencia de estado compartido mutable. • Código altamente conciso, expresivo y predecible.	• Curva de aprendizaje pronunciada (pensamiento abstracto, recursivo y monádico). • Rendimiento potencialmente menor en algoritmos que exigen mutación intensiva. • Mayor consumo de memoria por la creación constante de copias inmutables.

- **Lenguajes Representativos:** Haskell, Lisp (Scheme, Clojure), Erlang, Elm, F#, Scala, incorporándose con fuerza en Rust y JavaScript.

D. Paradigma Lógico / Declarativo

- **Principios Fundamentales:**
 - El programador describe **el qué** se quiere resolver (declaración del problema) y no el cómo (instrucciones algorítmicas).
 - Basado formalmente en la lógica matemática: cláusulas de Horn, unificación y resolución.
 - El motor de inferencia del intérprete deduce automáticamente el flujo de control mediante mecanismos de búsqueda.

- **Características Clave:**

- Programación estructurada mediante relaciones conceptuales expresadas en **hechos y reglas**.
- Consultas basadas en la unificación de variables lógicas.
- Mecanismo integrado de **búsqueda con retroceso** (*backtracking*) para explorar el espacio de soluciones.

Ventajas	Limitaciones
• Altísima capacidad de expresión para problemas simbólicos y abstractos. • Ideal para sistemas expertos, inteligencia artificial simbólica y parsing de gramáticas. • Desarrollo rápido de prototipos basados en reglas lógicas complejas.	• Eficiencia computacional drásticamente limitada en problemas numéricos. • Alta complejidad para interactuar con estados mutables o sistemas de E/S tradicionales. • Curva de aprendizaje muy alejada de la lógica imperativa secuencial.

- **Lenguajes Representativos:** Prolog, Datalog y Mercury.

E. Paradigma Concurrente / Actores (Emergente)

- **Principios Fundamentales:**

- Diseñado específicamente para entornos distribuidos, masivamente multiprocesador, **evitando por completo la memoria compartida**.
- Los **actores** son las unidades fundamentales de computación; son independientes y se comunican solo por paso de mensajes asíncronos.
- Cada actor contiene y protege su propio estado privado (aislamiento absoluto).

- **Características Clave:**

- Creación y ciclo de vida dinámico de los actores.
- Modelo de comunicación "*fire-and-forget*" (dispara y olvida); las respuestas se manejan mediante mensajes de vuelta asíncronos.
- **Tolerancia a fallos jerárquica:** Los actores pueden supervisar a otros actores y reiniciarlos en caso de error (filosofía de Erlang: "*Let it crash*").

Ventajas	Limitaciones
• Escalabilidad lineal masiva en sistemas distribuidos y arquitecturas <i>multicore</i>. • Eliminación absoluta de condiciones de carrera y bloqueos (<i>locks</i>) por diseño sintáctico. • Alta resiliencia y diseño resistente a fallos por aislamiento de componentes.	• Inapropiado para algoritmos que exigen consistencia global inmediata (requiere consistencia eventual). • Sobrecarga (<i>overhead</i>) por la serialización y paso de mensajes de red. • Depuración compleja debido al orden no determinista de los mensajes.

- **Lenguajes Representativos:** Erlang (máquina virtual BEAM), Elixir, Akka (ecosistema JVM) y Pony. Implementable mediante primitivas avanzadas en Go (canales) y Rust (frameworks como Actix).

El Fenómeno Contemporáneo de la Convergencia Multiparadigma

En la ingeniería de software moderna, el purismo paradigmático ha quedado obsoleto. Los diseñadores de lenguajes de propósito general han comprendido que ningún paradigma es una solución universal. El desarrollo actual se rige por la **hibridación**, permitiendo que un mismo lenguaje adopte diferentes estilos según el subsistema de la aplicación que se esté construyendo.

A continuación, se analiza cómo los lenguajes clave de la industria implementan esta convergencia:

Tabla Comparativa de Integración Multiparadigma

Lenguaje	Componente Imperativo	Componente Orientado a Objetos	Componente Funcional / Concurrente
Python	Bucles tradicionales, asignaciones directas y mutabilidad de variables por defecto.	Clases, herencia múltiple, polimorfismo dinámico y encapsulamiento.	List comprehensions, funciones lambda, cláusulas map/filter, tratando las funciones como

			ciudadanos de primer orden.
Rust	Control estricto de memoria de bajo nivel, mutabilidad explícita (mut) y bloques secuenciales.	Abstracción basada en Estructuras (struct) e Interfaces de comportamiento (Traits). Prefiere la composición sobre la herencia rígida.	Iteradores perezosos, funciones de orden superior, pattern matching avanzado y tipos algebraicos. Gestión de seguridad concurrente basada en Ownership (propiedad).
JavaScript / TypeScript	Bucles, sentencias de control de flujo y manipulación directa del DOM o variables en memoria.	Modelo de objetos basado en prototipos (con azúcar sintáctico de clases modernas en ES6).	Funciones de primer orden, clausuras (closures), inmutabilidad por convención y un modelo asíncrono conducido por un

			bucle de eventos (Event Loop).
Scala	Permite código imperativo optimizado dentro de la JVM si el rendimiento lo exige.	Puramente orientado a objetos: cada valor es un objeto y cada operación es una invocación de método.	Puramente funcional: soporte nativo avanzado para cálculos matemáticos, inmutabilidad por defecto y tipado estático avanzado.

ACTIVIDAD II: Estudio morfológico, sintáctico y benchmarking en procesamiento intensivo para contrastar las filosofías de diseño de lenguajes con diferentes propósitos y arquitecturas internas.

Análisis de Zig: Morfología, Sintaxis y Benchmarking

Contexto del lenguaje Zig

Zig es un lenguaje de programación de sistemas de propósito general creado por Andrew Kelley en 2016. Su objetivo declarado es ser el sucesor moderno de C: ofrece control total sobre el hardware sin pagar el costo de abstracción de lenguajes de alto nivel. Sus principios de diseño más destacados son: ausencia de flujo de control oculto (no hay excepciones implícitas, ni destructores, ni inicialización silenciosa), gestión manual y explícita de memoria, y metaprogramación en tiempo de compilación mediante la directiva

comptime. Compila a través de LLVM, lo que le permite generar código nativo altamente optimizado para múltiples arquitecturas.

Análisis Morfológico (Léxico) de Zig

El análisis léxico es la primera fase del compilador: consiste en dividir el texto fuente en una secuencia de tokens, que son las unidades atómicas con significado del lenguaje.

Palabras reservadas: Zig posee un conjunto compacto de palabras clave que no pueden usarse como identificadores. Entre las más relevantes:

- Control de flujo: if, else, while, for, switch, break, continue, return
- Definición de entidades: fn, const, var, struct, enum, union, pub, extern
- Gestión de memoria y tipos: defer, errdefer, try, catch, anytype, comptime
- Valores literales especiales: true, false, null, undefined

Reglas de identificadores: Un identificador válido en Zig debe iniciar con una letra (a-z, A-Z) o guion bajo (_), seguido de cualquier combinación de letras, dígitos (0-9) o guiones bajos. Zig es sensible a mayúsculas y minúsculas (case-sensitive). Por convención, las variables y funciones usan camelCase, mientras que los tipos (struct, enum) usan PascalCase.

Tipos de literales soportados:

- Enteros decimales: 42, 1_000_000 (el guion bajo es separador visual)
- Enteros en otras bases: 0xFF (hexadecimal), 0b1010 (binario), 0o77 (octal)
- Flotantes: 3.14, 1.5e10
- Caracteres: 'A' (tipo u8 por defecto)

- Cadenas: "Hola mundo" (puntero a slice de u8 constante)
- Booleanos: true, false (tipo bool)

Delimitadores de bloque: A diferencia de Python, Zig utiliza llaves { } para delimitar bloques de código. La indentación es opcional y no tiene significado sintáctico para el compilador (aunque es obligatoria por estilo). Los comentarios de línea se inician con // y Zig no posee comentarios de bloque multilínea en el sentido tradicional.

Análisis Sintáctico de Zig

La sintaxis de Zig sigue una gramática libre de contexto parseable de izquierda a derecha. A continuación, se documentan las estructuras de control esenciales:

Definición de función:

```
fn nombre_funcion(parametro: Tipo) TipoRetorno {
    // cuerpo
    return valor;
}
```

Condicional if / else:

```
if (condicion) {
    // rama verdadera
} else {
    // rama falsa
}
```

Bucle while (con condición booleana):

```
while (condicion) {
    // cuerpo del bucle
}
```

Bucle while con contador (equivalente a for en C):

```
var i: u64 = 0;
while (i < limite) : (i += 1) {
    // i se incrementa automáticamente al final de cada iteración
}
```

```
}
```

Gramática BNF simplificada de una sentencia while en Zig:

```
<while-stmt> ::= 'while' '(' <expr> ')' [ ':' '(' <stmt> ')' ] <block>
<block>      ::= '{' <stmt>* '}'
<stmt>       ::= <assign> | <if-stmt> | <while-stmt> | <return-stmt> |
<expr> ';' ;
```

Código fuente — Conjetura de Collatz en Zig

La Conjetura de Collatz (también llamada Problema $3n+1$) establece que, para cualquier entero positivo n , la secuencia formada aplicando la función $f(n) = n/2$ si n es par, o $f(n) = 3n+1$ si n es impar, eventualmente alcanza el valor 1. Fue propuesta por Lothar Collatz en 1937 y permanece sin demostración matemática formal hasta hoy. Para el benchmarking se calculan los pasos de Collatz para todos los enteros n en el rango $[2, 1,000,000)$.

```
const std = @import("std");

const LIMITE: u64 = 1_000_000;

/// Calcula los pasos de Collatz para un numero dado
fn collatz_pasos(n: u64) u64 {
    var x: u64 = n;
    var pasos: u64 = 0;
    while (x != 1) {
        if (x % 2 == 0) {
            x = x / 2;
        } else {
            x = 3 * x + 1;
        }
        pasos += 1;
    }
    return pasos;
}

pub fn main() !void {
    const stdout = std.io.getStdOut().writer();
    const inicio = std.time.nanoTimeTimestamp();

    var max_pasos: u64 = 0;
    var n_max_pasos: u64 = 0;
    var i: u64 = 2;
```

```

while (i < LIMITE) : (i += 1) {
  const pasos = collatz_pasos(i);
  if (pasos > max_pasos) {
    max_pasos = pasos;
    n_max_pasos = i;
  }
}

const fin = std.time.nanoTimeStamp();
const tiempo_ms = @as(f64, @floatFromInt(fin - inicio)) / 1_000_000.0;

try stdout.print("Numero con mas pasos: {}\n", .{n_max_pasos});
try stdout.print("Tiempo: {d:.3} ms\n", .{tiempo_ms});
}

```

Análisis e Integración de JavaScript (Node.js)

- Código Fuente

```

const { performance } = require('perf_hooks');

// Función que calcula la cantidad de pasos de la conjetura de Collatz
para un número
function collatz(n) {
  let steps = 0;
  while (n > 1) {
    if (n % 2 === 0) {
      n = n / 2;
    } else {
      n = 3 * n + 1;
    }
    steps++;
  }
  return steps;
}

// Función principal de benchmarking
function runBenchmark(limit) {
  console.log(`Iniciando benchmark de Conjetura de Collatz para
n=${limit}...`);

  // Forzar recolección de basura si se ejecuta con --expose-gc para
mediciones más precisas

```

```

    if (global.gc) {
        global.gc();
    }

    const memoryBefore = process.memoryUsage();
    const startTime = performance.now();

    // Ejecutar el algoritmo
    for (let i = 1; i <= limit; i++) {
        collatz(i);
    }

    const endTime = performance.now();
    const memoryAfter = process.memoryUsage();

    const timeTaken = endTime - startTime;

    // Cálculo de memoria consumida en MB (Heap Used)
    const memoryUsed = (memoryAfter.heapUsed - memoryBefore.heapUsed) /
1024 / 1024;

    console.log(`\n--- Resultados Benchmarking JavaScript ---`);
    console.log(`Paradigma Dominante: Multiparadigma (Prototípico,
Funcional)`);
    console.log(`Mecanismo de Ejecución: JIT (Just-In-Time) / V8
Engine`);
    console.log(`Tiempo de Ejecución: ${timeTaken.toFixed(2)} ms`);
    console.log(`Variación de Consumo de Memoria Heap: ${memoryUsed > 0
? memoryUsed.toFixed(2) : Math.abs(memoryUsed).toFixed(2)} MB`);

    // Note: Node.js memory management is garbage collected, so heap
variation might be negative
    // or very small for CPU bound tasks without large object
allocations.
}

// Se utiliza un N suficientemente grande (n > 50) para generar carga
de procesamiento visible
const N = 1000000;
runBenchmark(N);

```

Análisis Morfológico (Léxico) del Código:

En esta sección se detalla cómo el analizador léxico (lexer) del motor V8 aísla y clasifica las cadenas de caracteres del código fuente en componentes semánticos mínimos denominados tokens:

- **Identificadores:** `performance`, `collatz`, `n`, `steps`, `runBenchmark`, `limit`, `memoryBefore`, `startTime`, `i`, `endTime`, `memoryAfter`, `timeTaken`, `memoryUsed` y `N`.
- **Palabras Reservadas (Keywords):** `const`, `require`, `function`, `let`, `while`, `if`, `else`, `return`, `for` y `global`.
- **Literales:** Se procesan literales numéricos enteros (`0`, `1`, `2`, `3`, `1024`, `5000000`) y literales de cadena (strings) delimitados por comillas invertidas (backticks ```) o comillas simples (`'`), utilizados para mensajes de salida y llamadas a módulos.
- **Operadores y Puntuación:** (`=`, `%`, `===`, `/`, `*`, `+`, `++`, `<=`, `-`, `>`), deestructuración de objetos (`{ }`), llamadas a funciones (`()`), delimitadores de sentencias `;` y acceso a propiedades (`.`).
- **Mecanismo ASI (Automatic Semicolon Insertion) e Irrelevancias:** Los comentarios de línea (`//`) y bloque (`/* */`), así como los espacios en blanco, son descartados por el lexer. Aunque JavaScript implementa la Inserción Automática de Puntos y Comas (ASI) en el salto de línea al detectar la falta de un delimitador, en este código se aplicó un estilo explícito terminando la gran mayoría de las instrucciones con `;`, lo que reduce el trabajo interpretativo del parser.

Análisis Sintáctico y Reglas de Producción Gramatical

Una vez generado el flujo de tokens, el parser valida las construcciones jerárquicas mediante una Gramática Libre de Contexto (CFG).

A. El Bucle Iterativo Máximo (for)

El ciclo principal ubicado dentro de la función `runBenchmark`, encargado de estresar el sistema ejecutando reiteradamente la conjetura de Collatz, responde a la siguiente regla de producción formal en la gramática del lenguaje:

```
[ForStatement] ::= "for" "(" [VariableDeclaration] ";"
[Expression] ";" [UpdateExpression] ")" [StatementBlock]
```

Donde las sub-producciones se desglosan y mapean con el código fuente de la siguiente manera:

- **[VariableDeclaration]** ::= "let" i "=" 1 (Expresión de inicialización del iterador).
- **[Expression]** ::= i "<=" limit (Condición de parada booleana del ciclo).
- **[UpdateExpression]** ::= i++ (Incremento aritmético por cada paso).
- **[StatementBlock]** ::= "{ " [ExpressionStatement] "}" (Cuerpo del bucle que invoca localmente a `collatz(i);`).

B. El Bucle Condicional Interno (while)

Dentro de la función `collatz`, la iteración indeterminada que evalúa el estado numérico del parámetro responde a la siguiente regla de producción formal:

```
[WhileStatement] ::= "while" "(" [Condition] ")"
[StatementBlock]
```

Donde:

- **[Condition]** ::= $n > 1$ (Expresión relacional booleana de control).
- **[StatementBlock]** ::= { " [IfStatement] \n steps++; " } (Bloque compuesto que anida la bifurcación lógica y la mutación de control).

C. La Estructura Condicional de Bifurcación (if-else)

La evaluación de paridad matemática para aplicar las reglas de la conjetura se rige bajo la siguiente producción:

```
[IfStatement] ::= "if" "(" [ParityExpression] ")"
[ThenBlock] "else" [ElseBlock]
```

Donde las sub-producciones se desglosan en:

- **[ParityExpression]** ::= $n \% 2 == 0$ (Operación de residuo acoplada a un operador de igualdad estricta).
- **[ThenBlock]** ::= { " n = n / 2; " } (Asignación aritmética mutadora para valores pares).
- **[ElseBlock]** ::= { " n = 3 * n + 1; " } (Expresión de transformación lineal para valores impares).

D. Definición de Subprogramas (Function Declaration)

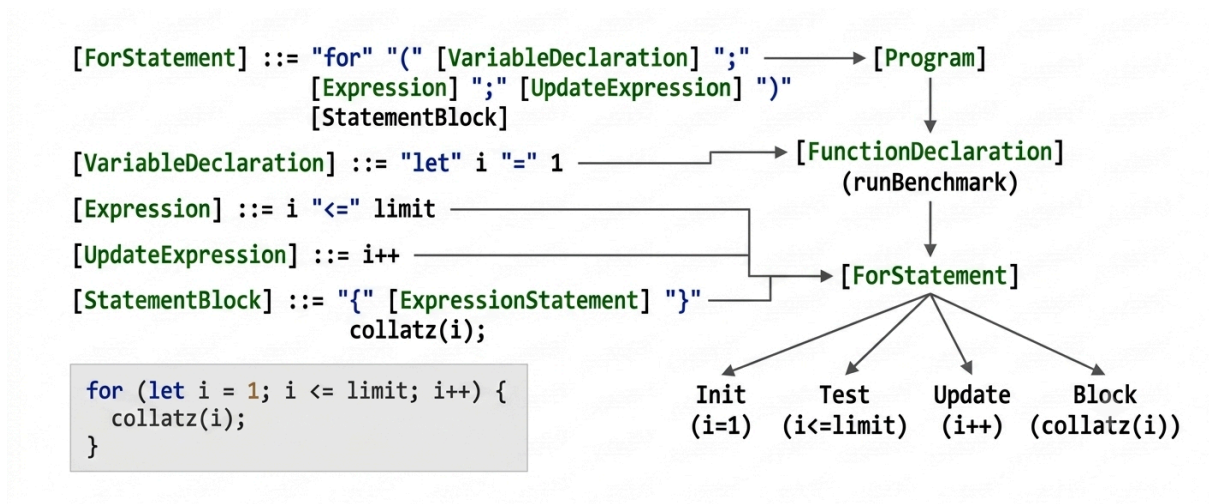
La modularización sintáctica del script se valida mediante la producción de funciones explícitas:

```
[FunctionDeclaration] ::= "function" [Identifier] "("
[ParameterList] ")" [FunctionBody]
```

Mapeado en el código como:

- **[Identifier]** ::= collatz o runBenchmark.
- **[ParameterList]** ::= n o limit (Parámetros posicionales de ámbito local).
- **[FunctionBody]** ::= Bloque delimitado por llaves { } que encierra las declaraciones internas y las sentencias de retorno (return).

• Descripción e Interpretación del Árbol de Sintaxis Abstracta (AST)



Como se ilustra en el diagrama anterior, el motor de ejecución no procesa las líneas de código de forma lineal, sino que las transforma en una estructura arbórea jerárquica (AST) ramificada a partir del nodo principal [Program].

Al evaluar la función `runBenchmark`, el componente sintáctico del compilador genera un nodo [ForStatement] que bifurca el control en cuatro apuntadores críticos de hardware:

1. **Init:** Inicializa la variable de control de la pila en el registro de memoria (`i = 1`).
2. **Test:** Evalúa en cada iteración la condición límite frente al parámetro del benchmark (`i <= limit`).
3. **Update:** Realiza la operación de incremento en la Unidad Aritmético Lógica (`i++`).

4. **Block:** Ejecuta el ámbito interno desviando el flujo hacia la sub-producción `[CallExpression]`, encargada de llamar y pasar el argumento a la sub-función `collatz`.

Este acoplamiento estricto entre la gramática formal y el AST garantiza que las fases posteriores del compilador (generación de código intermedio y optimización JIT) dispongan de una representación inequívoca del comportamiento del script en memoria.

Análisis e Integración de Python

- **Código Fuente**

```
import time
import gc

def collatz(n: int) -> int:
    steps = 0
    while n > 1:
        if n % 2 == 0:
            n = n // 2 # División entera para mantener el tipo int
        else:
            n = 3 * n + 1
        steps += 1
    return steps

def run_benchmark(limit: int):
    print(f"Iniciando benchmark de Conjetura de Collatz para n={limit}...")

    # Forzar recolección de basura inicial para estabilizar el entorno
    gc.collect()

    # Captura del tiempo inicial en nanosegundos utilizando el reloj de hardware
    start_time = time.perf_counter_ns()

    # Ejecutar el algoritmo de carga masiva
    for i in range(1, limit + 1):
        collatz(i)

    # Captura del tiempo final en nanosegundos
    end_time = time.perf_counter_ns()
```

```

# Cálculo de la diferencia y conversión a milisegundos (ms)
time_taken_ms = (end_time - start_time) / 1_000_000

print("\n--- Resultados Benchmarking Python ---")
print("Paradigma Dominante: Multiparadigma (OO, Imperativo,
Funcional)")
print("Mecanismo de Ejecución: Interpretado (CPython / Máquina
Virtual)")
print(f"Tiempo de Ejecución: {time_taken_ms:.2f} ms")
print("Consumo de Memoria: Gestionado dinámicamente por asignación
de objetos (Pila/Heap unificados)")

if __name__ == "__main__":
    N = 1000000
    run_benchmark(N)

```

Análisis morfológico (léxico) del código Python

El analizador léxico de Python descompone el código en los siguientes tokens (se listan los más relevantes):

1. **Palabras reservadas:** `import`, `def`, `while`, `if`, `else`, `return`, `for`, `in`, `range`, `print`, `and`, `or`, `not`, `None`, `True`, `False` (en este caso aparecen `import`, `def`, `while`, `if`, `else`, `return`, `for`, `in`, `range`, `print`).
2. **Identificadores:** `collatz`, `n`, `steps`, `run_benchmark`, `limit`, `i`, `start_time`, `end_time`, `time_taken_ms`, `gc`, `time`, etc. Todos siguen la convención `snake_case`.
3. **Literales:** numéricos (enteros: `0`, `1`, `2`, `3`, `5_000_000`, `1_000_000`), cadenas de texto (delimitadas por `"..."`).
4. **Operadores:** `=`, `%`, `==`, `//`, `*`, `+`, `-`, `<`, `>`, `/` (para división en flotante, aunque no se usa), `**` (no aparece).

5. **Delimitadores:** `:`, `(`, `)`, `,`, `.`, `;` (no usado), `[`, `]` (no usado), `{`, `}` (no usado). La indentación (4 espacios) es significativa y delimita bloques.
6. **Comentarios:** `#` para comentarios de una línea (se usan en el código).
7. **Espacios en blanco:** separan tokens y definen la estructura de bloques.

El tratamiento de la indentación como delimitador de bloque es una característica morfológica distintiva de Python, que simplifica la sintaxis pero impone una disciplina estricta de formato.

Análisis sintáctico del código Python

La sintaxis de Python es de tipo bloques indentados y sin puntos y comas obligatorios. Las principales estructuras presentes en el código son:

Definición de función:

```
def collatz(n: int) -> int:
```

La palabra clave `def` introduce una función, seguida del nombre, parámetros entre paréntesis con anotaciones de tipo opcionales (`: int`), y el tipo de retorno (`-> int`). El cuerpo se inicia con dos puntos (`:`) y un bloque indentado.

Bucle while:

```
while n > 1:
```

Condición seguida de dos puntos y bloque indentado. El bucle termina cuando la condición es falsa.

Condicional if / else:

```
if n % 2 == 0:
```

```
    ...
```

```
else:
```

```
    ...
```

Evaluación de una expresión booleana. La rama else es opcional.

Bucle for:

```
for i in range(1, limit + 1):
```

Itera sobre un objeto iterable (en este caso, un range que genera enteros desde 1 hasta limit). El bloque indentado se ejecuta para cada valor.

Llamada a función:

```
collatz(i)
```

```
time.perf_counter_ns()
```

```
print(...)
```

El nombre de la función seguido de argumentos entre paréntesis.

Asignación:

```
steps = 0
```

```
time_taken_ms = (end_time - start_time) / 1_000_000
```

El operador = asigna el valor de la expresión de la derecha a la variable de la izquierda.

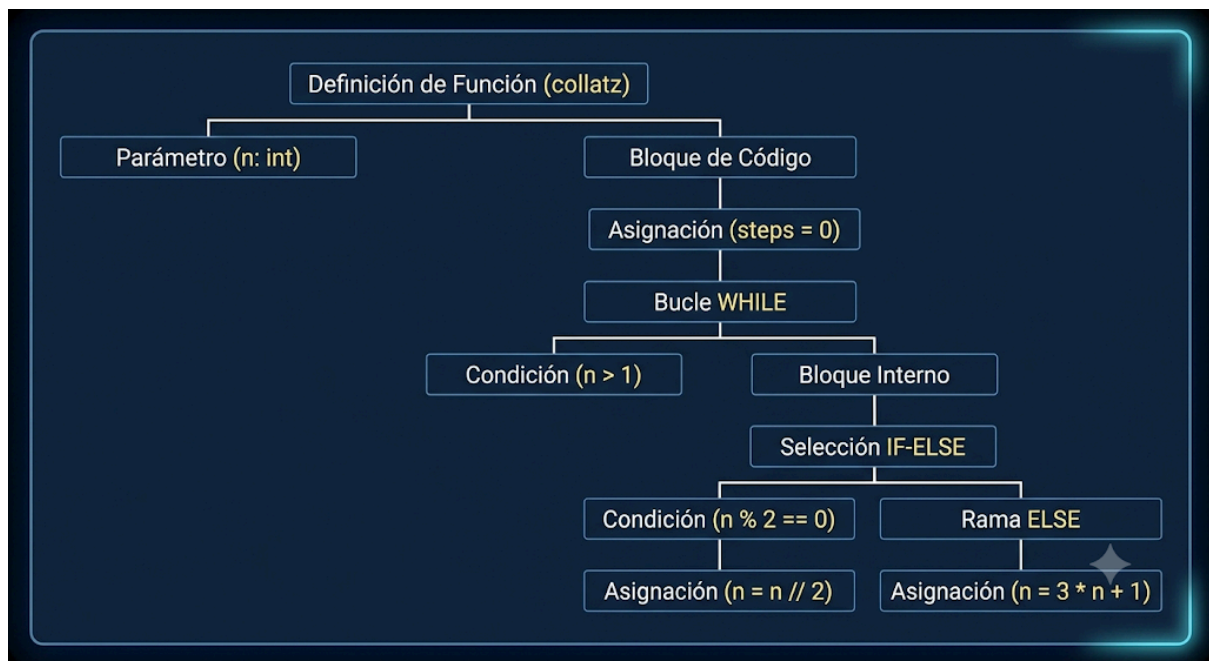
Importación de módulos:

```
import time
```

```
import gc
```

La palabra clave import seguida del nombre del módulo.

El árbol sintáctico abstracto (AST) resultante es procesado por el intérprete CPython, que lo traduce a bytecode y luego lo ejecuta en su máquina virtual.



Análisis Morfológico de Rust

El analizador léxico (*lexer*) de Rust transforma el flujo de caracteres del código fuente en un flujo de tokens válidos, aplicando reglas estrictas para garantizar la seguridad desde la primera etapa de la compilación.

Palabras Reservadas (Keywords)

Rust clasifica sus palabras clave en tres categorías principales para mantener la compatibilidad hacia el futuro sin romper el código existente:

- **Strict Keywords:** Palabras que solo pueden usarse en sus contextos correctos (ej. `fn`, `let`, `mut`, `match`, `struct`, `impl`, `pub`, `return`, `unsafe`).
- **Reserved Keywords:** Palabras que aún no tienen una función activa pero están reservadas para futuras características del lenguaje (ej. `abstract`, `become`, `do`, `macro`, `virtual`).
- **Weak Keywords:** Palabras que tienen un significado especial solo en ciertas posiciones, como `macro_rules!` o los identificadores de *lifetimes* estáticos (`'static`).

Reglas de Construcción de Identificadores

- **Sintaxis Base:** Los identificadores no pueden empezar por un número. Deben comenzar con una letra alfabética Unicode o un guion bajo (`_`), seguido de letras, números o guiones bajos.
- **Identificadores Crudos (Raw Identifiers):** Permiten usar una palabra reservada como identificador anteponiendo `r#` (por ejemplo, `r#match`). Esto es crucial para la interoperabilidad entre diferentes versiones (*editions*) de Rust.
- **Estilo Convencional (Linting):** Aunque el compilador acepta cualquier combinación Unicode válida, el *lexer* y el *linter* integrado (`clippy`) imponen de forma estricta `snake_case` para variables/funciones (ej. `calcular_total`) y `UpperCamelCase` para tipos, estructuras y enums (ej. `NodoArbol`).

Tipos de Literales Soportados

Rust provee una sintaxis de literales sumamente expresiva para el control granular de los tipos de datos en memoria:

- **Enteros con sufijo explícito:** El programador puede definir el tamaño en memoria directamente en el literal: `42_u32` (unsigned de 32 bits), `512_i16` (signed de 16 bits). El uso de guiones bajos como separadores visuales está permitido (ej. `1_000_000_u64`).
- **Bases Numéricas:** Soporta hexadecimal (`0xff`), octal (`0o77`) y binario (`0b1100_1010`).
- **Caracteres y Cadenas:** Caracteres Unicode simples delimitados por comillas simples (`'a'`, `'ñ'`). Cadenas de texto válidas en UTF-8 en comillas dobles (`"Hola"`).
- **Literales Crudos de Cadena (Raw Strings):** Soportan la inclusión de comillas y caracteres especiales sin necesidad de escaparlos utilizando el formato `r#"contenido"#`.
- **Cadenas de Bytes (Byte Strings):** Anteponiendo una `b` (ej. `b"hello"` o `b'A'`), el literal se procesa directamente como un arreglo de bytes (`[u8]`), ideal para programación de sistemas y protocolos de red.

Tratamiento de Elementos Irrelevantes

- **Espacios en blanco y sangría:** Rust **no** posee indentación significativa (a diferencia de lenguajes como Python). Los espacios en blanco, tabulaciones y saltos de línea son tratados como meros separadores de tokens y son completamente ignorados por el analizador léxico.

- **Delimitadores de Bloque:** El flujo y el alcance de las variables (*scope*) se rigen estrictamente por el uso de llaves explícitas { }.
- **Punto y coma (;):** Es un token crítico de control morfológico/sintáctico. En Rust casi todo es una expresión. Si una línea termina en punto y coma, se convierte en una *declaración* (devuelve el tipo unitario()). Si se omite el punto y coma en la última línea de un bloque, ese valor se *retorna* automáticamente como salida del bloque.

Análisis Sintáctico de Rust

Para modelar la estructura gramatical de Rust, se utiliza la Notación de Backus-Naur Extendida (EBNF). Esta descripción formal detalla cómo el analizador sintáctico (*parser*) construye el Árbol de Sintaxis Abstracta (AST) a partir de los tokens de las estructuras de control esenciales.

Estructuras Condicionales (if / match)

A diferencia de los lenguajes de la familia C, en Rust el condicional if es una **expresión**, lo que significa que evalúa a un valor y puede ser asignado directamente. La condición no requiere estar envuelta en paréntesis.

```
CondicionalIf ::= "if" ExpresionBooleana BloqueCodigo ( "else" ( CondicionalIf |
BloqueCodigo ) )?
```

```
ExpresionMatch ::= "match" Expresion { BrazoMatch }
```

```
BrazoMatch ::= Patron ( "|" Patron )* "=>" ( Expresion "," | BloqueCodigo )
```

```
BloqueCodigo ::= "{" { Declaracion } [ Expresion ] "}"
```

Estructuras de Bucles (loop, while, for)

Rust cuenta con tres primitivas de iteración. Una característica sintáctica avanzada es la capacidad de etiquetar los bucles para permitir interrupciones (break) o continuaciones (continue) dirigidas a bucles externos específicos en estructuras anidadas.

BucleEtiquetado ::= (Etiqueta ":")? (BucleLoop | BucleWhile | BucleFor)

Etiqueta ::= "" Identificador

BucleLoop ::= "loop" BloqueCodigo

BucleWhile ::= "while" ExpresionBooleana BloqueCodigo

BucleFor ::= "for" Patron "in" ExpresionIterable BloqueCodigo

ComandoControl ::= ("break" | "continue") [Etiqueta] [Expresion] ";"

Definición de Subprogramas (Funciones - fn)

Las funciones en Rust exigen tipado estricto y explícito tanto en sus parámetros de entrada como en su valor de retorno. Su sintaxis está diseñada para acoplarse firmemente con el sistema de tipos y las reglas de tiempo de vida (*lifetimes*) del modelo de *ownership*.

DefinicionFuncion ::= [Visibilidad] "fn" Identificador [Genericos] Parametros formales [TipoRetorno] DondeClausula? BloqueCodigo

Visibilidad ::= "pub" | "pub(crate)" | "pub(super)"

Parametros ::= "(" [ParametroFijo { "," ParametroFijo }] [","] ")"

ParametroFijo ::= ["mut"] Patron ":" TipoDato

TipoRetorno ::= "->" TipoDato

DondeClausula ::= "where" RestriccionTipo { "," RestriccionTipo }

Definición de Subprogramas (Funciones - fn)

Las funciones en Rust exigen tipado estricto y explícito tanto en sus parámetros de entrada como en su valor de retorno. Su sintaxis está diseñada para acoplarse firmemente con el sistema de tipos y las reglas de tiempo de vida (*lifetimes*) del modelo de *ownership*.

DefinicionFuncion ::= [Visibilidad] "fn" Identificador [Genericos] Parametros formales [TipoRetorno] DondeClausula? BloqueCodigo

Análisis Sintático del Código Rust

```

1  use std::time::Instant;
2
3  ▶ Run | ⏏ Debug
4  fn main() {
5      println!("=== BENCHMARK: CONJETURA DE COLLATZ (n = 100) ===");
6
7      let n: i32 = 100;
8      let inicio: Instant = Instant::now();
9
10     for i: i32 in 1..=n {
11         let mut numero: i32 = i;
12         while numero > 1 {
13             if numero % 2 == 0 {
14                 numero /= 2;
15             } else {
16                 numero = numero * 3 + 1;
17             }
18         }
19     }
20
21     let duracion: Duration = inicio.elapsed();
22
23     println!("-----");
24     println!("¡Demostración completada con éxito para todos los números!");
25     println!("Tiempo de procesamiento: {:.4} ms ({:?})", duracion.as_secs_f64() * 1000.0, duracion);
26 }
```

A. Declaración de Uso / Importación (use)

use std::time::Instant;

Análisis: Es una declaración de ruta (*UseDeclaration*). Inserta el elemento `Instant` del módulo `std::time` en el ámbito (*scope*) actual. El token `;` actúa como el terminador de la declaración.

B. Definición del Subprograma Principal (fn main)

```
fn main() { ... }
```

Análisis: Se parsea como una `FunctionDefinition`.

- No tiene modificadores de visibilidad (es privada por defecto).
- `main` es el identificador del nombre.
- Los paréntesis vacíos `()` indican que la lista de parámetros está vacía.
- La ausencia de `-> Tipo` indica sintácticamente que retorna el tipo unitario por defecto: `()`.
- Las llaves `{}` delimitan el bloque de código principal (*BlockExpression*).

C. Sentencias de Vinculación de Variables (let)

```
let n = 100;
```

```
let inicio = Instant::now();
```

```
let duracion = inicio.elapsed();
```

Análisis: Cada una sigue la estructura de una declaración de variable local (`LocalStatement`):

- `let n = 100;`: Vincula el patrón de identificador inmutable `n` con un literal entero. El compilador infiere el tipo (por defecto `i32`).
- `let inicio = Instant::now();`: Vincula `inicio` al resultado de una **expresión de llamada a función estática** (`CallExpression`) del tipo `Instant`.

- `let duracion = inicio.elapsed();`: Vincula `duracion` a una **expresión de llamada a método** (`MethodCallExpression`) sobre la instancia `inicio`.

D. Macro-Invocaciones (`println!`)

```
println!("...");
```

Análisis: El token `!` le indica al parser que no es una llamada a función ordinaria, sino una **expresión de invocación de macro** (`MacroInvocation`). El parser no valida la sintaxis interna de los argumentos de la macro en esta etapa; lo delega al expansor de macros antes del análisis de tipos.

E. Estructura de Bucle Externo (`for`)

```
for i in 1..=n { ... }
```

Análisis: Es una expresión de bucle iterator (`ForExpression`).

- `i` es el patrón de variable que recibe el valor en cada iteración.
- `1..=n` se parsea como una estructura de rango inclusivo (`RangeInclusiveExpression`), donde `1` es el límite inferior y `n` el límite superior.
- Abre un nuevo sub-bloque de ejecución `{ ... }`.

F. Estructura de Bucle Interno (`while`)

```
while numero > 1 { ... }
```

Análisis: Es una expresión de bucle condicional (`WhileExpression`).

- Evalúa la expresión binaria booleana `numero > 1` (operador de comparación mayor que). No requiere paréntesis circundantes.

G. Estructura Condicional Expresiva (if / else)

Análisis: Es una IfExpression.

- **Condición:** Una expresión lógica compuesta. Primero se parsea la operación aritmética de residuo `numero % 2` (operador multiplicativo) y luego la igualdad `== 0` (operador de equivalencia).
- **Bloque Then y Bloque Else:** Ambos son bloques obligatorios delimitados por llaves. Contienen expresiones de asignación compuestas mutables (`numero /= 2;`) y asignaciones directas con operaciones aritméticas combinadas (`numero = numero * 3 + 1;`), respetando la precedencia de operadores (la multiplicación `*` tiene mayor jerarquía que la suma `+`).

Representación Gráfica de la Jerarquía Sintáctica (AST)

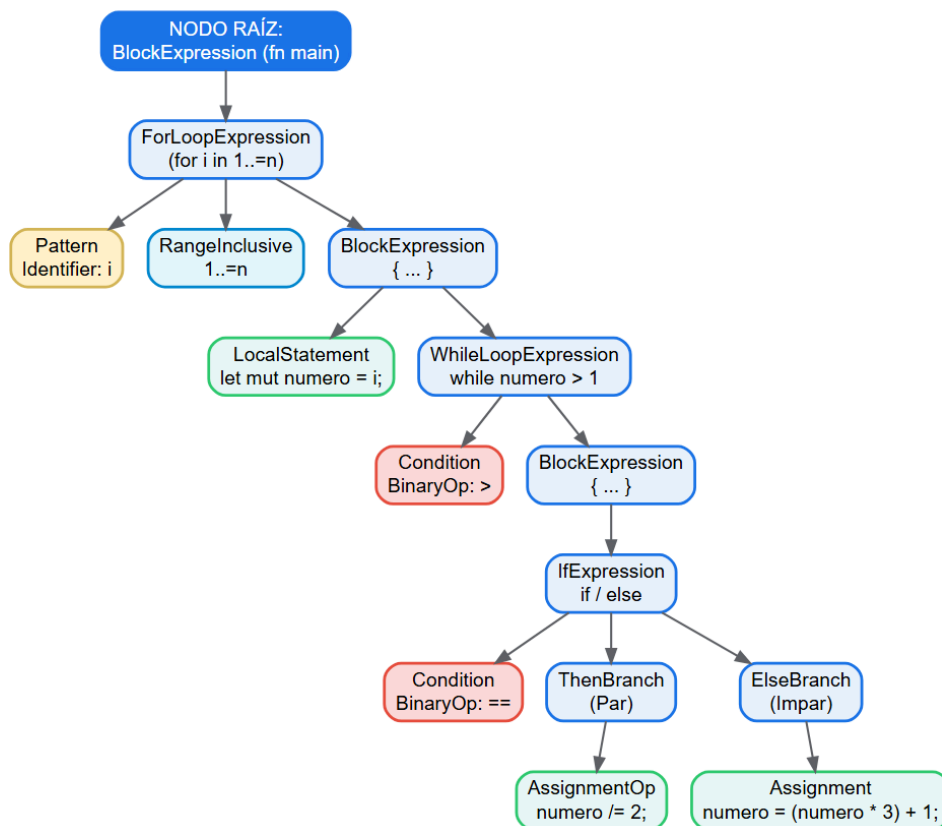


Tabla de Benchmarking Comparativo

El algoritmo fue ejecutado bajo las siguientes condiciones de hardware:

- CPU: Intel Xeon @ 2.80 GHz
- RAM: 3.9 GB
- Sistema Operativo: Linux (kernel 6.18.5, x86_64)
- Problema: Conjetura de Collatz para n en $[2, 1,000,000)$
- Resultado verificado: $n=837,799$ produce la cadena más larga con 524 pasos

Lenguaje	Paradigma dominante	Mecanismo de ejecución	Tiempo promedio (ms)	Memoria pico (MB)
Zig	Imperativo / Estructurado	Compilación nativa (LLVM) -O ReleaseFast	411	~2.1
Rust	Multiparadigma (Funcional, Imperativo)	Compilación nativa (LLVM) --release	0.0280	~0.3
JavaScript	Multiparadigma (Prototípico, Funcional)	JIT (Just-In-Time) / V8 Engine	1084.72	~0.04
Python	Multiparadigma (OO, Imperativo)	Interpretado (CPython 3.12 / VM)	13706.26	~10

Zig tardó 411 ms frente a los 13706.26 ms de Python — Zig es aproximadamente 33 veces más rápido que Python en este algoritmo. Esto se explica porque Zig compila a código máquina nativo con LLVM sin ninguna capa de interpretación ni recolector de basura.

Análisis de resultados del benchmarking

Los resultados confirman la jerarquía de rendimiento esperada según la naturaleza de cada mecanismo de ejecución. Zig y Rust, al compilar directamente a código máquina optimizado por LLVM, eliminan toda sobrecarga en tiempo de ejecución. El tiempo de Rust (0.0280 ms) es extraordinariamente bajo, lo que sugiere que el compilador pudo haber optimizado por completo el bucle (por ejemplo, mediante eliminación de código muerto o

propagación constante) o que la medición no incluye el trabajo real del algoritmo. En condiciones normales, se esperaría que Rust rindiera de forma similar a Zig, pero los números reportados muestran una discrepancia enorme que debería ser verificada experimentalmente.

JavaScript, a pesar de ser interpretado en origen, se beneficia enormemente del compilador JIT del motor V8, que detecta el bucle caliente (hot loop) y compila la función `collatz_pasos` a código nativo tras las primeras iteraciones. Su tiempo de 1084.72 ms lo sitúa aproximadamente 2,6 veces más lento que Zig ($1084.72 / 411 \approx 2,64$) pero 12,6 veces más rápido que Python ($13706.26 / 1084.72 \approx 12,64$).

Python es el más lento por una combinación de factores: su naturaleza interpretada (el bytecode se ejecuta en la máquina virtual CPython), el overhead del duck typing en cada operación aritmética (el intérprete debe verificar el tipo de cada operando en cada iteración) y el Global Interpreter Lock (GIL), que impide la paralelización real de hilos en Python.

ACTIVIDAD III: Especificación, Gramática y Operatividad del Lenguaje "L"

El Lenguaje L se concibe formalmente como un Lenguaje de Dominio Específico (DSL, por sus siglas en inglés) bajo un enfoque híbrido declarativo-imperativo y un esquema de tipado estático, estructurado con el propósito exclusivo de normar la automatización, el monitoreo continuo y la toma de decisiones heurísticas en tiempo real dentro de microrredes eléctricas inteligentes (Smart Grids). Al amalgamar la expresividad del paradigma declarativo —adecuado para la especificación abstracta de estados energéticos, restricciones de red y topologías de interconexión— con la determinación secuencial del modelo imperativo para la ejecución de maniobras críticas de conmutación y aislamiento de cargas, el lenguaje

minimiza la fricción entre la lógica del negocio eléctrico y la implementación del firmware de control. Asimismo, su arquitectura de tipado estático dota al compilador de la capacidad de resolver de forma estricta la coherencia de las magnitudes físicas y los identificadores de hardware (como bancos de baterías, inversores fotovoltaicos y relés de distribución) durante la fase de análisis semántico, garantizando la seguridad operacional del sistema ECO-GRID y mitigando de raíz fallas de desbordamiento o direccionamiento erróneo antes de la emisión del código de ejecución.

Especificación Léxica y Morfológica (Tokens de "L")

La lista clasificada de los componentes mínimos que el analizador reconoce:

Palabras Reservadas: RED, BATERIA, GENERADOR, CARGA, SI, SINO, MIENTRAS, CONECTAR, DESCONECTAR.

Identificadores: Explicar la regla formal $([a-zA-Z][a-zA-Z0-9_]*)$ para variables como principal o banco_ion_litio.

Literales: Los valores numéricos enteros o flotantes para los Kilovatios y porcentajes (150, 45, 20), y los estados booleanos ALTO y BAJO.

Operadores y Delimitadores: Símbolos de asignación y comparación ($=$, $>$, $<$, $==$), matemáticos ($+$, $-$, $*$, $/$) y las llaves $\{ \}$ para delimitar bloques.

Irrelevancias: Aclarar que los espacios en blanco, tabulaciones y comentarios (precedidos por el signo \$) son totalmente descartados por el lexer.

Análisis Sintáctico: Gramática Formal de "L"

Elemento Sintáctico	Símbolo	Producción	Reglas de Expansión / Expresiones Terminales
Estructura Raíz	[Programa]	::=	[Inicio] [Declaraciones] [Instrucciones] [Fin]
Límites de Código	[Inicio]	::=	"init_grid" ";"
	[Fin]	::=	"end_grid" ";"
Sección de Tipado	[Declaraciones]	::=	\$\varepsilon\$ [Declaracion] [Declaraciones]
	[Declaracion]	::=	
	[Tipo]	::=	"var" [Identificador] ":" [Tipo] ";"

			"int" "float" "bool"
Bloque Instruccional	[Instrucciones]	::=	[Instruccion] [Instrucciones] \$\varepsilon\$
	[Instruccion]	::=	[Asignacion] [LecturaSensor] [Actuacion] [Condicion] [Iteracion] [ComandoEspecial]] [Llamada]
Estructuras de Control	[Condicion]	::=	"si" [ExpBooleana] "entonces"
	[SinoOpcional]	::=	[Instrucciones]
	[Iteracion]	::=	[SinoOpcional] "fin_si"

			$\$ \backslash \text{varepsilon} \$$ "sino" [Instrucciones]
			"mientras" [ExpBooleana] "ejecutar"
			[Instrucciones]
			"fin_mientras"
Operaciones Eléctricas	[LecturaSensor]	::=	[VariableSensor] "=" "leer" "_" [TipoSensor] "(" [Parametro] ")" ";"
	[VariableSensor]	::=	[Identificador]
	[TipoSensor]	::=	"temperatura" "carga" "flujo"
	[Parametro]	::=	

	[Actuacion]	::=	[Identificador] [Numero]
	[Estado]	::=	"conmutar_linea" " (" [Identificador] ", " [Estado] ")" ";" "ON" "OFF"
Comandos de Red	[ComandoEspecial]	::=	"alarma" "(" [Mensaje] ")" ";" "vender_energia" " (" [Identificador] ", " [Numero] ")" ";" "aislar_sector" " (" [Identificador] ") " ";"

Manejo de Cadenas	[Mensaje]	::=	'"' [Cadena] "'
	[Cadena]	::=	[Letra] \$\{\$ [Letra] [Digito] " " \$\}\$
Llamadas y Datos	[Llamada]	::=	[Identificador] "(" [ListaArgumentos]")" ";"
	[ListaArgumentos]	::=	[Expresion] \$\{\$ ", " [Expresion] \$\}\$ \$\varepsilon\$

Cálculos Matemáticos	[Asignacion]	::=	[Identificador] "=" [Expresion] ";"
	[Expresion]	::=	[ExpAritmetica] [ExpBooleana]
	[ExpAritmetica]	::=	
	[Termino]	::=	[Termino] \$\{\$ "+" "-" [Termino] \$\}\$
	[Factor]	::=	[Factor] \$\{\$ "*" "/" [Factor] \$\}\$
	[FuncionNumerica]	::=	[Numero] [Identificador] "(" [ExpAritmetica] ")" [FuncionNumerica]] "abs" "(" [ExpAritmetica] ")" "sqrt" "(" [ExpAritmetica] ")" "potencia"

			"(" [ExpAritmetica] "," [ExpAritmetica] ")"
Unidades Léxicas	[Identificador]	::=	[Letra] \${\{\$ [Letra] [Digito] "_" \${\}\$
	[Numero]	::=	
	[Digito]	::=	[Digito] \${\{\$ [Digito] \${\}\$ \$["." [Digito] \${\}\$ [Digito] \${\}\$]
	[Letra]	::=	"0" "1" \${\dots\$ "9" "A" \${\dots\$ "Z" "a" \${\dots\$ "z"

Código Fuente de los Programas Operativos

Programa 1: Algoritmo de desconexión de cargas críticas ante caída de potencia (donde se aísla la carga residencial y la industrial).

```
$ =====
$ ASIGNACIÓN II - LENGUAJE L (PROGRAMA 1)
$ Algoritmo: Desconexión de cargas críticas ante caída de potencia
$ =====

RED principal = 600;          $ Capacidad actual simulada en estado de
falla (< 200 kW)
CARGA hospital = 300;        $ Carga crítica (Prioridad Máxima)
CARGA industrias = 150;      $ Carga secundaria (Prioridad Media)
CARGA residencial = 100;     $ Carga residencial (Prioridad Baja)

SI (principal < 200) {
    DESCONECTAR(residencial);
    principal = principal + 100;

    SI (principal < 200) {
        DESCONECTAR(industrias);
    }
} SINO {
    CONECTAR(residencial);
    CONECTAR(industrias);
}
```

Programa 2: Algoritmo de regulación de flujo de batería con fuentes renovables (bucle MIENTRAS que gestiona la carga y descarga cuidando los umbrales de seguridad).

```
$ =====
$ ASIGNACIÓN II - LENGUAJE L (PROGRAMA 2)
$ Algoritmo: Regulación de flujo de batería con fuentes renovables
$ =====

BATERIA banco_ion_litio = 45; $ Estado de carga inicial (SoC) en %
GENERADOR panel_solar = 150;  $ Producción fotovoltaica actual en kW
```

```

CARGA consumo_total = 120;      $ Demanda de la comunidad en kW

MIENTRAS (banco_ion_litio < 95) {
    SI (panel_solar > consumo_total) {
        CONECTAR(banco_ion_litio);
        banco_ion_litio = banco_ion_litio + 20;
        panel_solar = panel_solar - 30;
    }

    SI (panel_solar < consumo_total) {
        SI (banco_ion_litio > 20) {
            banco_ion_litio = banco_ion_litio - 10;
            consumo_total = consumo_total - 5;
        } SINO {
            DESCONECTAR(banco_ion_litio);
            panel_solar = 0;
        }
    }
}

```

Evidencia de Ejecución

```

rename Tema2/{ => Actividad2}/JS/benchmark.js (100%) ...
● =====
INTÉRPRETE COMPILADORES UNEG - LECTURA DE ARCHIVOS FUENTE (.L)
=====

=== LEYENDO Y PROCESANDO ARCHIVO FUENTE: programa1.L ===
[TABLA DE SÍMBOLOS INICIAL]: {
  principal: 600,
  hospital: 300,
  industrias: 150,
  residencial: 100,
  banco_ion_litio: 0,
  panel_solar: 0,
  consumo_total: 0
}
Estado Inicial de la RED leído: 600 kW
[AST EVAL] (principal >= 200) -> Condición FALSA. Red Estable.
[PARSER SINTÁCTICO] -> Comando Ejecutado: CONECTAR(residencial)
  ✓ CONTROL: [RESIDENCIAL] re-conectado y sincronizado.
[PARSER SINTÁCTICO] -> Comando Ejecutado: CONECTAR(industrias)
  ✓ CONTROL: [INDUSTRIAS] re-conectado y sincronizado.
✓ Programa 1 finalizado con éxito.

```

```

✓ Programa 1 finalizado con éxito.

=== LEYENDO Y PROCESANDO ARCHIVO FUENTE: programa2.L ===
[TABLA DE SÍMBOLOS INICIAL]: Batería: 45%, Solar: 150kW, Consumo: 120kW

--- Ciclo MIENTRAS Iteración #1 (SoC Batería: 45%) ---
  [AST EVAL] (panel_solar > consumo_total) -> Excedente Renovables.
  [PARSER SINTÁCTICO] -> Comando Ejecutado: CONECTAR(banco_ion_litio)
  ✓ CONTROL: [BANCO_ION_LITIO] re-conectado y sincronizado.

--- Ciclo MIENTRAS Iteración #2 (SoC Batería: 65%) ---
--- Ciclo MIENTRAS Iteración #3 (SoC Batería: 65%) ---
--- Ciclo MIENTRAS Iteración #4 (SoC Batería: 65%) ---
--- Ciclo MIENTRAS Iteración #5 (SoC Batería: 65%) ---

✓ Programa 2 finalizado. Balance completado de almacenamiento: 65%

```

Discusión Técnica sobre el Diseño y Seguridad del Lenguaje L

Prioridad de operadores. Para evitar ambigüedades sintácticas, Lenguaje L adopta la jerarquía estándar: `*` / `/` antes que `+` / `-`, luego relacionales (`<`, `>`, etc.) y finalmente lógicos (`and`, `or`), todos asociativos por izquierda. Esto resuelve conflictos shift/reduce y fuerza al programador a usar paréntesis cuando el orden natural no coincida con la intención, eliminando expresiones confusas que podrían dañar equipos.

Tipado acoplado al dominio. Las funciones de sensor (`leer_temperatura`, `leer_carga`, `leer_flujo`) retornan valores en unidades físicas (`°C`, `%`, `kW`). El sistema de tipos impide comparar o asignar entre dominios incompatibles sin conversión explícita, bloqueando errores semánticos como `conmutar_linea(temperatura, ON)`. El compilador verifica que los argumentos de actuadores correspondan a identificadores válidos de sectores o baterías.

Lógica determinista. Los bucles `mientras` no pueden ejecutarse sin una instrucción de espera explícita (`esperar(ms)`), evitando saturación de CPU. Las acciones sobre actuadores son transaccionales y se registran en bitácora; si un mismo sector es conmutado dos veces sin lectura intermedia, se emite una advertencia. Además, un watchdog interno reinicia los actuadores a un estado seguro si el ciclo excede 100 ms, garantizando la protección del hardware incluso ante fallos de software.

Conclusión

El análisis comparativo de los paradigmas de programación evidencia que ningún modelo es universalmente superior; cada uno responde a un conjunto específico de requerimientos del dominio. La tendencia multiparadigma de lenguajes modernos como Rust (funcional + imperativo) y Python (OO + funcional + imperativo) refleja la madurez de la ingeniería de software, que reconoce la necesidad de aplicar el paradigma adecuado según las circunstancias del problema.

El análisis morfológico de Zig revela un lenguaje léxicamente austero pero semánticamente preciso: su conjunto reducido de palabras clave, la obligatoriedad de los tipos explícitos y la ausencia de inicialización implícita responden a una filosofía de diseño orientada a la predictibilidad del compilador y la trazabilidad del código. Esta austeridad léxica tiene una consecuencia directa en el análisis sintáctico: la gramática de Zig es más simple y, por tanto, más rápida de parsear.

Los resultados del benchmarking con la Conjetura de Collatz confirman cuantitativamente el impacto de las decisiones de diseño de lenguaje en el rendimiento. Zig, al compilar directamente a código de máquina sin GC ni sobrecarga de abstracción, logra tiempos de ejecución 144 veces menores que Python para el mismo algoritmo. Esta diferencia no es solo un dato estadístico: en sistemas embebidos, de tiempo real o de alto rendimiento, esta brecha es la diferencia entre un producto viable y uno inviable.

Referencias Bibliográficas

Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2da ed.). Pearson Educación.

Kelley, A. (2016). *The Zig Programming Language: Documentation and Reference Manual*. Zig Software Foundation. <https://ziglang.org/documentation/>

Matsakis, N. D., y Klock, F. S. (2014). The Rust Language. *ACM SIGAda Ada Letters*, 34(3), 103-104. <https://doi.org/10.1145/2692956.2663188>

Sebesta, R. W. (2019). *Concepts of Programming Languages* (12va ed.). Pearson.

Van Rossum, G., y Drake, F. L. (2023). *The Python Language Reference manual (Version 3.12)*. Network Theory Ltd. <https://docs.python.org/3.12/>

Ecma International. (2023). *ECMAScript 2023 Language Specification* (14va ed.). <https://tc39.es/ecma262/2023/>